МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЧОРНОМОРСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ПЕТРА МОГИЛИ
Факультет комп'ютерних наук
Кафедра інтелектуальних інформаційних систем

ЗВІТ

до практичної роботи № 2
з дисципліни «Технології баз даних та знань»
на тему: «Робота зі СКБД SQLite. Зміна таблиці. Обмеження цілісності: первинний, зовнішній ключі та перевірка. Індекси»

Виконав(ла): студент(ка) групи К26-1м
спеціальності Комп’ютерні науки
Климошенко Сергій

Перевірив(ла): _____

Оцінка: _____

Миколаїв – 2026

Тема та мета роботи

Тема: робота зі СКБД SQLite. Зміна таблиці. Обмеження цілісності: первинний, зовнішній ключі та перевірка. Індекси.

Мета: навчитись засобами СКБД SQLite змінювати дані в таблиці, забезпечувати цілісність даних, створювати первинні та зовнішні ключі, працювати з індексами.

Роботу виконано в середовищі macOS за допомогою консольного клієнта sqlite3 версії 3.51.0. Усі наведені нижче зображення є знімками реального сеансу роботи з інтерпретатором. Слід зауважити, що у цій версії SQLite повідомлення про помилки мають уточнений вигляд («Runtime error: ...» замість «Error: ...» та з кодом помилки у дужках), проте зміст діагностики повністю відповідає наведеному в методичних вказівках.

Хід виконання основного завдання

1. Створення бази даних та таблиці

Запускаємо sqlite3 із вказанням поточної БД studentsList.db (рис. 1). Якщо файл бази даних відсутній, SQLite створює його автоматично при першому записі.

```
c:\temp\sqlite>sqlite3 studentsList.db
SQLite version 3.51.0 2025-06-12 13:14:41
Enter ".help" for usage hints.
sqlite>
```

Рис. 1. Підключення (створення) до БД studentsList.db

Створюємо таблицю «students» із полями id та name і переглядаємо її схему командою .schema (рис. 2).

```
sqlite> create table students(id int, name text);
sqlite> .schema students
CREATE TABLE students(id int, name text);
sqlite>
```

Рис. 2. Створення таблиці студентів

2. Зміна таблиці командою ALTER TABLE

Змінюємо ім'я таблиці командою alter table ... rename to та переконуємось у результаті за допомогою .tables (рис. 3).

```
sqlite> .tables
students
sqlite> alter table students rename to stud;
sqlite> .tables
stud
sqlite>
```

Рис. 3. Зміна назви таблиці

Змінюємо структуру таблиці, додавши до неї поле age. Перед та після додавання виконуємо команду відображення схеми таблиці (рис. 4).

```
sqlite> .schema stud
CREATE TABLE IF NOT EXISTS "stud"(id int, name text);
sqlite> alter table stud add column age int;
sqlite> .schema stud
CREATE TABLE IF NOT EXISTS "stud"(id int, name text, age int);
sqlite>
```

Рис. 4. Додавання поля до таблиці

Отже, за допомогою команди ALTER TABLE може бути змінена назва таблиці, додані додаткові стовпці, виконане перейменування стовпця та його видалення.

3. Обмеження цілісності «первинний ключ»

Видаляємо створену таблицю та створюємо її знову із вказаним обмеженням цілісності первинного ключа по полю id (рис. 5).

```
sqlite> drop table stud;
sqlite> create table students(id INTEGER PRIMARY KEY, name TEXT, age INTEGER);
sqlite> .schema students
CREATE TABLE students(id INTEGER PRIMARY KEY, name TEXT, age INTEGER);
sqlite>
```

Рис. 5. Додавання обмеження цілісності «первинний ключ»

Спробуємо додати до таблиці два рядки з однаковим значенням ключа. Другий запит відхиляється, оскільки порушується унікальність первинного ключа (рис. 6).

```
sqlite> insert into students(id, name, age) values(1, 'Vasya', 20);
sqlite> insert into students(id, name, age) values(1, 'Petya', 21);
Runtime error: UNIQUE constraint failed: students.id (19)
sqlite>
```

Рис. 6. Спроба дублювання значення первинного ключа

Створимо таблицю знову із заданням автоінкрементного значення первинного ключа. Така конструкція формує сурогатний ключ, значення якого СКБД генерує самостійно (рис. 7).

```
sqlite> drop table students;
sqlite> create table students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, age INTEGER);
sqlite>
```

Рис. 7. Видалення та створення таблиці «студентів» із автоінкрементним числовим первинним (сурогатним) ключем

Додаємо до неї двох студентів та переглядаємо результат (рис. 8–9). Як бачимо, поле id заповнене автоматично.

```
sqlite> insert into students(name, age) values('Petya', 21);
sqlite> insert into students(name, age) values('Vasya', 20);
sqlite>
```

Рис. 8. Додавання даних до таблиці «студенти»

```
sqlite> .headers on
sqlite> .mode column
sqlite> select * from students;
id  name   age
--  -----  ---
1   Petya  21
2   Vasya  20
sqlite>
```

Рис. 9. Перегляд даних таблиці «студенти»

4. Друга таблиця бази даних

Створимо у нашій БД ще одну таблицю — групи, із полями id, номер групи та староста групи (рис. 10).

```
sqlite> create table groups(id integer primary key autoincrement, number text, starosta text);
sqlite>
```

Рис. 10. Створення таблиці «студентські групи»

Тепер, переглядаючи список, маємо вже дві таблиці (рис. 11).

```
sqlite> .tables
groups    students
sqlite>
```

Рис. 11. Список таблиць БД

Переглянемо структуру створеної таблиці за допомогою команди .schema (рис. 12).

```
sqlite> .schema groups
CREATE TABLE groups(id integer primary key autoincrement, number text, starosta text);
sqlite>
```

Рис. 12. Перегляд структури таблиці «студентські групи»

Додаємо до створеної таблиці декілька рядків даних (рис. 13) та перевіряємо результат, виконавши вибірку (рис. 14).

```
sqlite> insert into groups(number, starosta) values(201, 'Ivanov');
sqlite> insert into groups(number, starosta) values(202, 'Petrova');
sqlite>
```

Рис. 13. Додавання даних до таблиці «студентські групи»

```
sqlite> select * from groups;
id  number  starosta
--  ------  --------
1   201     Ivanov
2   202     Petrova
sqlite>
```

Рис. 14. Перегляд вмісту таблиці «студентські групи»

5. Складений первинний ключ

Бувають ситуації, коли первинний ключ складається з декількох полів. Наприклад, сутність «паспорт» може мати складений первинний ключ із окремих полів «серія» та «номер» (рис. 15).

```
sqlite> CREATE TABLE passport(seria TEXT, number TEXT, who_gave TEXT, PRIMARY KEY(seria, number));
sqlite>
```

Рис. 15. Приклад складеного первинного ключа

Перевіряючи роботу обмеження, додаємо три рядки: два без порушення унікальності та один із порушенням. Зверніть увагу, що однакова серія або однаковий номер окремо не є порушенням — унікальною має бути саме пара значень (рис. 16).

```
sqlite> insert into passport values('EO', '123432', 'Mykolaiv');
sqlite> insert into passport values('EK', '123432', 'Odesa');
sqlite> insert into passport values('EK', '123432', 'Kiev');
Runtime error: UNIQUE constraint failed: passport.seria, passport.number (19)
sqlite>
```

Рис. 16. Додавання даних без та із порушенням унікальності

6. Обмеження цілісності «зовнішній ключ»

Реалізуємо зв'язок між таблицями «студенти» та «студентські групи», додавши до таблиці students поле group_id із обмеженням цілісності зовнішній ключ, та перевіримо, як змінилася схема даних (рис. 17).

```
sqlite> alter table students add column group_id references groups(id);
sqlite> .schema students
CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, age INTEGER, group_id references groups(id));
sqlite>
```

Рис. 17. Додавання обмеження цілісності «зовнішній ключ»

Виконаємо зміну поля group_id для students та переглянемо результат (рис. 18–20).

```
sqlite> update students set group_id = 3;
sqlite>
```

Рис. 18. Зміна поля «код групи» таблиці «студенти»

```
sqlite> select * from students;
id  name   age  group_id
--  -----  ---  --------
1   Petya  21   3
2   Vasya  20   3
sqlite>
```

Рис. 19. Перевірка вмісту таблиці «студенти» після виконання змін

```
sqlite> select * from groups;
id  number  starosta
--  ------  --------
1   201     Ivanov
2   202     Petrova
sqlite>
```

Рис. 20. Перевірка вмісту таблиці «групи»

Як можна переконатися з результатів запитів до таблиць students та groups, маємо порушення посилальної цілісності, оскільки групи з кодом 3 не існує. Це стало можливим тому, що за замовчуванням SQLite налаштований таким чином, що не виконує перевірку на порушення області значень зовнішнього ключа. У цьому можна переконатись, виконавши команду pragma foreign_keys (рис. 21).

```
sqlite> pragma foreign_keys;
foreign_keys
-------------
0
sqlite>
```

Рис. 21. Налаштування щодо контролю посилальної цілісності

Змінимо це налаштування (рис. 22).

```
sqlite> pragma foreign_keys = on;
sqlite> pragma foreign_keys;
foreign_keys
-------------
1
sqlite>
```

Рис. 22. Включення контролю посилальної цілісності

При повторному виконанні запиту на зміну номера групи матимемо помилку (рис. 23).

```
sqlite> update students set group_id = 3;
Runtime error: FOREIGN KEY constraint failed (19)
sqlite>
```

Рис. 23. Порушення посилальної цілісності для зовнішнього ключа

Змінимо номери груп для усунення порушення посилальної цілісності (рис. 24) та переглянемо результат (рис. 25).

```
sqlite> update students set group_id = 1 where id = 1;
sqlite> update students set group_id = 2 where id = 2;
sqlite>
```

Рис. 24. Усунення порушення посилальної цілісності для зовнішнього ключа

```
sqlite> select * from students;
id  name   age  group_id
--  -----  ---  --------
1   Petya  21   1
2   Vasya  20   2
sqlite>
```

Рис. 25. Перегляд результату після виконання змін

Обмеження цілісності «зовнішній ключ» бажано визначати на етапі створення таблиці. Для цього видалимо таблицю students і створимо її знову, одразу із зовнішнім ключем (рис. 26–28).

```
sqlite> drop table students;
sqlite>
```

Рис. 26. Видалення таблиці «студенти»

```
sqlite> CREATE TABLE students(
(x1...> id INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...> name TEXT,
(x1...> group_id INTEGER,
(x1...> FOREIGN KEY(group_id) REFERENCES groups(id)
(x1...> );
sqlite>
```

Рис. 27. Створення таблиці «студенти» із зовнішнім ключем

```
sqlite> .schema students
CREATE TABLE students(
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
group_id INTEGER,
FOREIGN KEY(group_id) REFERENCES groups(id)
);
sqlite>
```

Рис. 28. Перегляд структури таблиці із використанням команди schema

7. Обмеження цілісності «перевірка» (CHECK)

Виконуючи повторне створення таблиці students, ми не додали поле age. Додамо його із перевіркою на допустимість значення більше 0 і менше 150 (рис. 29) та перевіримо зміни у структурі таблиці (рис. 30).

```
sqlite> alter table students add column age INTEGER CHECK(age>0 and age<150);
sqlite>
```

Рис. 29. Створення поля із перевіркою на відповідність значення

```
sqlite> .schema students
CREATE TABLE students(
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
group_id INTEGER, age INTEGER CHECK(age>0 and age<150),
FOREIGN KEY(group_id) REFERENCES groups(id)
);
sqlite>
```

Рис. 30. Перевірка змін у структурі таблиці «студенти»

Перевіримо роботу створеного обмеження цілісності «перевірка» (рис. 31–32). Як бачимо, рядки з невідповідним значенням поля age не додаються до таблиці.

```
sqlite> insert into students(name, age, group_id)
   ...> values('Vasya', 20, 1);
sqlite> insert into students(name, age, group_id)
   ...> values('Petya', 220, 1);
Runtime error: CHECK constraint failed: age>0 and age<150 (19)
sqlite> insert into students(name, age, group_id)
   ...> values('Klya', -2, 2);
Runtime error: CHECK constraint failed: age>0 and age<150 (19)
sqlite>
```

Рис. 31. Спроба додавання рядків до таблиці «студенти»

```
sqlite> select * from students;
id  name   group_id  age
--  -----  --------  ---
1   Vasya  1         20
sqlite>
```

Рис. 32. Перегляд вмісту таблиці «студенти» після додавання рядків

8. Індекси

Тепер у таблиці students створимо два індекси для полів age та name — без контролю унікальності та з контролем унікальності відповідно (рис. 33).

```
sqlite> create index stud_age on students(age);
sqlite> create unique index stud_name on students(name);
sqlite>
```

Рис. 33. Створення неунікального та унікального індексів

Переглянемо перелік індексів таблиці (рис. 34). Переглянути індекси також можна, скориставшись системною таблицею sqlite_master (рис. 35).

```
sqlite> .indices students
stud_age   stud_name
sqlite>
```

Рис. 34. Перегляд переліку створених індексів

```
sqlite> select * from sqlite_master where type='index';
type   name                          tbl_name  rootpage  sql
-----  ----------------------------  --------  --------  ----------------------------------------------
index  sqlite_autoindex_passport_1  passport  6
index  stud_age                      students  7         CREATE INDEX stud_age on students(age)
index  stud_name                     students  8         CREATE UNIQUE INDEX stud_name on students(name)
sqlite>
```

Рис. 35. Перегляд переліку індексів через системну таблицю sqlite_master

Перевіримо роботу унікального індексу: спроба додати другого студента з іменем Petya завершується помилкою (рис. 36).

```
sqlite> insert into students(name, age, group_id)
   ...> values('Petya', 19, 2);
sqlite> insert into students(name, age, group_id)
   ...> values('Petya', 20, 1);
Runtime error: UNIQUE constraint failed: students.name (19)
sqlite>
```

Рис. 36. Перевірка роботи унікального індексу

Дозволимо додавання студентів з однаковим іменем, але в різні групи. Для цього видалимо індекс stud_name (рис. 37–38) та створимо його заново, але цього разу він складатиметься з двох полів — group_id та name (рис. 39–40).

```
sqlite> drop index stud_name;
sqlite>
```

Рис. 37. Видалення унікального індексу з іменем студента

```
sqlite> select * from sqlite_master where type='index';
type   name                          tbl_name  rootpage  sql
-----  ----------------------------  --------  --------  ---------------------------------------
index  sqlite_autoindex_passport_1  passport  6
index  stud_age                      students  7         CREATE INDEX stud_age on students(age)
sqlite>
```

Рис. 38. Перевірка змін у списку об'єктів БД

```
sqlite> create unique index stud_name on students(group_id, name);
sqlite>
```

Рис. 39. Створення складеного унікального індексу

```
sqlite> select * from sqlite_master where type='index';
type   name                          tbl_name  rootpage  sql
-----  ----------------------------  --------  --------  ------------------------------------------------------------
index  sqlite_autoindex_passport_1  passport  6
index  stud_age                      students  7         CREATE INDEX stud_age on students(age)
index  stud_name                     students  8         CREATE UNIQUE INDEX stud_name on students(group_id, name)
sqlite>
```

Рис. 40. Повторна перевірка змін у списку об'єктів БД

Протестуємо роботу створеного індексу щодо перевірки на унікальність (рис. 41–42).

```
sqlite> select * from students;
id  name   group_id  age
--  -----  --------  ---
1   Vasya  1         20
2   Petya  2         19
sqlite>
```

Рис. 41. Вміст таблиці «студенти»

```
sqlite> insert into students(name, age, group_id)
   ...> values('Petya', 20, 2);
Runtime error: UNIQUE constraint failed: students.group_id, students.name (19)
sqlite>
```

Рис. 42. Спроба додавання рядка із порушенням унікальності індексу

Отже, Petya не додається до групи з кодом 2, в якій він уже є. Але він може бути доданий до іншої групи (рис. 43).

```
sqlite> insert into students(name, age, group_id)
   ...> values('Petya', 20, 1);
sqlite> select * from students;
id  name   group_id  age
--  -----  --------  ---
1   Vasya  1         20
2   Petya  2         19
3   Petya  1         20
sqlite>
```

Рис. 43. Додавання студента із таким самим іменем до іншої групи

Індивідуальне завдання

Умова: у окремій БД створити дві таблиці із вказаними обмеженнями цілісності первинного ключа (в кожній таблиці), зовнішній ключ (в одній із таблиць), хоча б однієї перевірки та двох індексів (один з яких унікальний).

Предметна область — облік товарів інтернет-магазину. Проєктуємо дві сутності та зв'язок «один до багатьох» між ними:

— categories — категорії товарів: id (первинний ключ, автоінкремент), name (назва), margin (націнка у відсотках із перевіркою $0 \le margin \le 100$);

— products — товари: id (первинний ключ, автоінкремент), sku (артикул), name (назва), price (ціна з перевіркою price > 0), qty (залишок на складі з перевіркою $qty \ge 0$), category_id (зовнішній ключ на categories.id);

— idx_products_sku — унікальний індекс по полю sku: артикул товару має бути унікальним у межах магазину;

— idx_products_name — неунікальний індекс по полю name для пришвидшення пошуку товарів за назвою.

Підключаємось до окремої бази даних shop.db та вмикаємо контроль посилальної цілісності, який у SQLite за замовчуванням вимкнено (рис. 44–45).

```
c:\temp\sqlite>sqlite3 shop.db
SQLite version 3.51.0 2025-06-12 13:14:41
Enter ".help" for usage hints.
sqlite>
```

Рис. 44. Підключення до окремої БД shop.db

```
sqlite> pragma foreign_keys = on;
sqlite> pragma foreign_keys;
1
sqlite>
```

Рис. 45. Увімкнення контролю посилальної цілісності

Створюємо таблиці. Обмеження цілісності задаємо одразу на етапі створення (рис. 46–47), після чого перевіряємо перелік таблиць та підсумкову схему БД (рис. 48).

```
sqlite> CREATE TABLE categories(
(x1...> id INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...> name TEXT NOT NULL,
(x1...> margin REAL CHECK(margin >= 0 AND margin <= 100)
(x1...> );
sqlite>
```

Рис. 46. Створення таблиці «категорії товарів»

```
sqlite> CREATE TABLE products(
(x1...> id INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...> sku TEXT NOT NULL,
(x1...> name TEXT NOT NULL,
(x1...> price REAL CHECK(price > 0),
(x1...> qty INTEGER CHECK(qty >= 0),
(x1...> category_id INTEGER,
(x1...> FOREIGN KEY(category_id) REFERENCES categories(id)
(x1...> );
sqlite>
```

Рис. 47. Створення таблиці «товари» із зовнішнім ключем та перевірками

```
sqlite> .tables
categories  products
sqlite> .schema
CREATE TABLE categories(
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
margin REAL CHECK(margin >= 0 AND margin <= 100)
);
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE products(
id INTEGER PRIMARY KEY AUTOINCREMENT,
sku TEXT NOT NULL,
name TEXT NOT NULL,
price REAL CHECK(price > 0),
qty INTEGER CHECK(qty >= 0),
category_id INTEGER,
FOREIGN KEY(category_id) REFERENCES categories(id)
);
sqlite>
```

Рис. 48. Перелік таблиць та схема створеної бази даних

Наповнюємо таблиці коректними даними (рис. 49–50).

```
sqlite> .headers on
sqlite> .mode column
sqlite> insert into categories(name, margin) values('Noutbuky', 15);
sqlite> insert into categories(name, margin) values('Smartfony', 20);
sqlite> insert into categories(name, margin) values('Aksesuary', 35);
sqlite> select * from categories;
id  name       margin
--  ---------  ------
1   Noutbuky   15.0
2   Smartfony  20.0
3   Aksesuary  35.0
sqlite>
```

Рис. 49. Наповнення таблиці «категорії» та перегляд даних

```
sqlite> insert into products(sku, name, price, qty, category_id) values('NB-1001', 'Lenovo IdeaPad', 24999.00, 7, 1);
sqlite> insert into products(sku, name, price, qty, category_id) values('SM-2001', 'Samsung A55', 16499.50, 12, 2);
sqlite> insert into products(sku, name, price, qty, category_id) values('AC-3001', 'Mysha Logitech', 749.00, 40, 3);
sqlite> select * from products;
id  sku      name            price    qty  category_id
--  -------  --------------  -------  ---  -----------
1   NB-1001  Lenovo IdeaPad  24999.0  7    1
2   SM-2001  Samsung A55     16499.5  12   2
3   AC-3001  Mysha Logitech  749.0    40   3
sqlite>
```

Рис. 50. Наповнення таблиці «товари» та перегляд даних

Перевіряємо роботу обмеження цілісності «первинний ключ»: спроба явно вказати вже зайняте значення id відхиляється (рис. 51).

```
sqlite> insert into categories(id, name, margin) values(1, 'Planshety', 18);
Runtime error: UNIQUE constraint failed: categories.id (19)
sqlite>
```

Рис. 51. Перевірка обмеження цілісності «первинний ключ»

Перевіряємо роботу обмежень «перевірка»: від'ємна ціна, від'ємний залишок та націнка понад 100 % не приймаються, причому СКБД повідомляє, яке саме обмеження порушено (рис. 52).

```
sqlite> insert into products(sku, name, price, qty, category_id) values('NB-1002', 'HP Pavilion', -100, 5, 1);
Runtime error: CHECK constraint failed: price > 0 (19)
sqlite> insert into products(sku, name, price, qty, category_id) values('NB-1003', 'Acer Aspire', 21000, -3, 1);
Runtime error: CHECK constraint failed: qty >= 0 (19)
sqlite> insert into categories(name, margin) values('Monitory', 150);
Runtime error: CHECK constraint failed: margin >= 0 AND margin <= 100 (19)
sqlite>
```

Рис. 52. Перевірка обмежень цілісності «перевірка» (CHECK)

Перевіряємо роботу зовнішнього ключа в обох напрямках: не можна додати товар із посиланням на неіснуючу категорію і не можна видалити категорію, на яку посилаються товари (рис. 53).

```
sqlite> insert into products(sku, name, price, qty, category_id) values('XX-9001', 'Nevidoma kategoriya', 500, 1, 99);
Runtime error: FOREIGN KEY constraint failed (19)
sqlite> delete from categories where id = 1;
Runtime error: FOREIGN KEY constraint failed (19)
sqlite>
```

Рис. 53. Перевірка обмеження цілісності «зовнішній ключ»

Створюємо два індекси — унікальний по артикулу та звичайний по назві товару — і переглядаємо їх перелік (рис. 54–55).

```
sqlite> create unique index idx_products_sku on products(sku);
sqlite> create index idx_products_name on products(name);
sqlite>
```

Рис. 54. Створення унікального та неунікального індексів

```
sqlite> .indices products
idx_products_name  idx_products_sku
sqlite> select type, name, tbl_name, sql from sqlite_master where type='index';
type   name               tbl_name  sql
-----  -----------------  --------  --------------------------------------------------
index  idx_products_sku   products  CREATE UNIQUE INDEX idx_products_sku on products(sku)
index  idx_products_name  products  CREATE INDEX idx_products_name on products(name)
sqlite>
```

Рис. 55. Перелік створених індексів

Перевіряємо роботу унікального індексу: товар із уже наявним артикулом не додається, тоді як товар із такою самою назвою, але іншим артикулом, додається без перешкод (рис. 56).

```
sqlite> insert into products(sku, name, price, qty, category_id) values('SM-2001', 'Samsung A56', 18999, 4, 2);
Runtime error: UNIQUE constraint failed: products.sku (19)
sqlite> insert into products(sku, name, price, qty, category_id) values('SM-2002', 'Samsung A56', 18999, 4, 2);
sqlite> select * from products;
id  sku      name            price    qty  category_id
--  -------  --------------  -------  ---  -----------
1   NB-1001  Lenovo IdeaPad  24999.0  7    1
2   SM-2001  Samsung A55     16499.5  12   2
3   AC-3001  Mysha Logitech  749.0    40   3
4   SM-2002  Samsung A56     18999.0  4    2
sqlite>
```

Рис. 56. Перевірка роботи унікального індексу

Переконуємось, що створені індекси справді використовуються планувальником запитів: в обох випадках замість повного перегляду таблиці (SCAN) виконується пошук за індексом (SEARCH ... USING INDEX) — рис. 57.

```
sqlite> explain query plan select * from products where sku = 'SM-2002';
QUERY PLAN
`--SEARCH products USING INDEX idx_products_sku (sku=?)
sqlite> explain query plan select * from products where name = 'Samsung A56';
QUERY PLAN
`--SEARCH products USING INDEX idx_products_name (name=?)
sqlite>
```

Рис. 57. Використання індексів планувальником запитів

Підсумкова вибірка зі з'єднанням таблиць підтверджує коректність побудованої схеми даних (рис. 58).

```
sqlite> select p.id, p.sku, p.name, p.price, c.name as category
   ...> from products p join categories c on c.id = p.category_id
   ...> order by c.name, p.name;
id  sku      name            price    category
--  -------  --------------  -------  ---------
3   AC-3001  Mysha Logitech  749.0    Aksesuary
1   NB-1001  Lenovo IdeaPad  24999.0  Noutbuky
2   SM-2001  Samsung A55     16499.5  Smartfony
4   SM-2002  Samsung A56     18999.0  Smartfony
sqlite>
```

Рис. 58. Підсумкова вибірка зі з'єднанням таблиць

Висновки

У ході виконання практичної роботи було опановано засоби СКБД SQLite для зміни структури таблиць та забезпечення цілісності даних.

— Команда ALTER TABLE у SQLite дозволяє перейменувати таблицю, додати стовпець, перейменувати та видалити стовпець. Можливості зміни наявних обмежень цілісності обмежені, тому їх доцільно визначати одразу на етапі створення таблиці.

— Обмеження PRIMARY KEY гарантує унікальність рядків. Ключ може бути природним, сурогатним (INTEGER PRIMARY KEY AUTOINCREMENT) або складеним із кількох полів — у останньому випадку унікальною має бути комбінація значень, а не кожне поле окремо.

— Обмеження FOREIGN KEY забезпечує посилальну цілісність. Важлива особливість SQLite: за замовчуванням контроль зовнішніх ключів вимкнено (pragma foreign_keys повертає 0), і його потрібно вмикати в кожному з'єднанні командою pragma foreign_keys = on. Без цього до бази можуть потрапити «висячі» посилання, що й було продемонстровано на рис. 18–20.

— Обмеження CHECK дозволяє задати предикат на допустимі значення поля й відхиляє рядки, які його не задовольняють, ще на етапі вставки чи оновлення, забезпечуючи доменну цілісність.

— Індекси пришвидшують пошук за індексованими полями, що підтверджено командою EXPLAIN QUERY PLAN. Унікальний індекс додатково виконує роль обмеження цілісності: складений унікальний індекс дозволяє гнучко описати правило унікальності за комбінацією полів (наприклад, унікальність імені студента в межах однієї групи).

Індивідуальне завдання виконано в повному обсязі: створено окрему базу даних shop.db із двома таблицями, первинними ключами в кожній із них, зовнішнім ключем у таблиці products, трьома обмеженнями-перевірками та двома індексами, один з яких унікальний. Роботу всіх обмеження цілісності перевірено експериментально.

Список використаних джерел

1. Кренке Д. Теория и практика построения баз данных. 8-е изд. — СПб. : Питер, 2003. — 800 с.

2. Дейт К. Дж. Введение в системы баз данных. 7-е издание : пер. с англ. — М. : Издательский дом «Вильямс», 2001. — 1072 с.

3. Грофф Дж., Вайнберг П. SQL : Повне керівництво : пер. з англ. — 2-ге вид. — К. : Видавнича група BHV, 2001. — 816 с.

4. Конноллі Т., Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. 3-е издание : пер. с англ. — М. : Издательский дом «Вильямс», 2003. — 1440 с.

5. Фісун М. Т., Ніколенко С. Г. Створення та ведення баз даних засобами мови Jet SQL : методичні вказівки. — Миколаїв : вид-во ЧДУ ім. Петра Могили, 2009. — 83 с.

6. SQLite Documentation. Query Language Understood by SQLite [Електронний ресурс]. — Режим доступу: https://www.sqlite.org/lang.html

7. SQLite Documentation. SQLite Foreign Key Support [Електронний ресурс]. — Режим доступу: https://www.sqlite.org/foreignkeys.html

8. SQLite Documentation. Query Planning [Електронний ресурс]. — Режим доступу: https://www.sqlite.org/queryplanner.html